The Session Manager Field Manual

A warm, plain-English guide to AI agents — and to running your own small agency of them with Claude Code and Session Manager. No experience needed.

Version 2.4.0 · released 2026-11-06 · documents Session Manager v0.129.0

Contents

Agents, explained

1. Welcome — your friendly map of what's ahead
2. What is an AI agent?
3. Claude Code in plain English

Meet your agency

1. Meet your agency — the team that works for you
2. Your first Session — from idea to working agent
3. Work that runs while you're away — plans and the Scheduler
4. Trust, but verify — how the work gets checked

Make it yours

1. Shaping your team — Agents, Missions and one-click Actions
2. House rules — instructions, settings and permissions
3. New abilities — skills, plugins, tools and hooks
4. Around the cockpit — Home, memory, usage, history and more
5. Good habits — great results, safely

Reference

1. Glossary — every term in plain English

Welcome — your friendly map of what's ahead

This manual is a friendly introduction to AI agents, and to Session Manager, the free app that gives you a small team of them. It answers one question: what are you actually working with, and how do you get the most out of it? No experience needed — just curiosity.

Who this manual is for

You're the right reader if you're curious about Claude Code and want to understand what's going on before you dive in. You don't need to know how to code. You don't need any technical background. If a word here might be new to you, this manual explains it the first time it shows up, or points you to the glossary.

Maybe you've heard the word "agent" and wondered what it means. Maybe you already use Claude Code and want a friendlier way to work with it. Either way, start here, and read forward.

This manual teaches rather than just shows. It explains why something works before it shows you how to use it. Learning the clicks alone only helps until the next update; understanding the idea behind them helps you find your way around long after that.

Claude Code, in one breath

Claude Code is a command-line tool from Anthropic. You give it a goal in plain English, and it plans the steps, uses tools, and works through them until the goal is done.

You'll usually meet it inside a terminal — a plain text window where you type a request and read the reply. That might sound intimidating if you've never used one. You don't have to face it alone: giving Claude Code a friendlier home is exactly what Session Manager is for, which is the next thing this chapter covers.

That's the whole idea in one breath. The next chapter, What is an AI agent?, slows down and explains why that's such a different thing from a chatbot that only answers questions.

What Session Manager adds

Session Manager is a free desktop app. It gives Claude Code a friendly home to live in, and it lets you run a small team of these agents at once, instead of just one.

> **Think of it like this:**
>
> Session Manager turns you into the director of your own small agency. You decide what gets built. An agent called the architect plans the work and breaks it into pieces. Builders called dev-leads each take one piece and build it. Another agent, the validator, checks the finished work against what you asked for. And a work calendar called the Scheduler lines it all up and runs it, even while you're away from your keyboard.

You'll meet each of these roles properly in Part Two. For now, the point is simple: instead of one assistant, you get a small team, and Session Manager is the room they all work in.

Every one of those agents runs the same real Claude Code underneath — Session Manager doesn't replace it or change what it can do. It wraps it in a cockpit instead: a window with tabs for your projects, a calendar that can run work while you're away, and one place to see everything your agency is doing at once.

How this manual is organized

The manual has three parts, plus a glossary you can open anytime. Each part builds on the one before it.

Agents, explained — what an AI agent actually is, and how Claude Code works, in plain English. Continues at What is an AI agent?

→

Meet your agency — the team Session Manager gives you, how to run your first piece of work, and how that work gets checked. Starts at Meet your agency.

→

Make it yours — how to write your own agents, set your house rules, and get the most from the rest of the app. Starts at Shaping your team.

After that comes the Glossary — every term this manual uses, in plain English, in one place.

How to read it

Read the three parts in order the first time through. Each one leans on ideas from the last, so skipping ahead means meeting a term before it's explained.

The glossary works differently. It isn't a chapter you read once — it's a lookup you keep open. Hit an unfamiliar word anywhere in this manual, and the glossary is one click away.

Each chapter is short enough to read in one sitting, so there's no need to rush through the whole manual at once. Come back whenever you have a few minutes, pick up where you left off, and lean on the glossary for anything that's slipped your mind.

> **Good to know:**
>
> Session Manager itself is free. Download the installer for your computer from `https://bilko.run/products/session-manager`. Nothing here asks you to pay for the app, and this manual is free too.

What you learned

- This manual assumes no prior experience — every term gets explained along the way.
- Claude Code is a command-line tool from Anthropic that plans and works through steps toward a goal you give it.
- Session Manager is a free desktop app that gives Claude Code a home and lets you run a small team of agents.
- The manual has three parts — Agents, explained; Meet your agency; and Make it yours — plus a glossary you can open anytime.
- Read the three parts in order, and use the glossary as a lookup whenever you need it.

Next up: What is an AI agent?

What is an AI agent?

A chatbot answers a question and stops. An agent takes a goal, does the work itself, and keeps going until the job is finished. This chapter shows you the difference, and the four parts every agent is built from.

A chatbot answers. An agent acts.

A chatbot listens to you, thinks, and replies with words. That's the whole exchange. It never leaves the conversation to touch anything in the real world.

An agent is different. Give it a goal, and it can look at your files, change them, and run commands on your computer. Claude Code — the command-line tool this manual is about — works this way. It reads your project, edits files, and runs commands to get real work done.

That's a bigger deal than it sounds. A chatbot might describe how to fix a bug in words. An agent opens the file, fixes the bug, runs the tests, and tells you it's done.

The agent loop: look, think, act, check, repeat

An agent doesn't finish a job in one step. It moves through a loop, again and again, until the job is actually done.

Look. It reads what's there right now — your files, a command's output, or an error message.

→

Think. It decides what to do next, based on the goal you gave it.

→

Act. It makes a move: edits a file, runs a command, or reads something new.

→

Check. It looks at what happened. Did that work? Is the goal met yet?

→

Repeat. If the job isn't done, it loops back to Look and keeps going.

This loop is why an agent can carry out a real task instead of just answering a single question. It keeps working, step by step, until it reaches the goal or needs your help.

The four parts of every agent

Every AI agent, including Claude Code, is built from four parts. Picture a new employee on their first day, and each part matches something you'd hand them.

Part	What it does
Model	The brain. It reads the goal and decides what to do next.
Tools	The hands. They let the agent read files, write files, and run commands.
Instructions	The job description. Your rules and goals, written down for the agent to follow.
Context and memory	The notebook and desk. Everything the agent can currently see and recall.

> **Think of it like this:**
> a new hire needs a brain to make decisions, hands to do the work, a job description to know what's expected, and a desk with notes to track where they left off. Take away any one part, and the other three can't finish the job alone.

Context and memory deserve a closer look, because they have a limit. An agent can only hold so much on its desk at once — file contents, your instructions, and everything said so far in the conversation. When that space fills up, older material gets summarized to make room. That's why a long conversation can sometimes act like it's forgotten something you said an hour ago.

What agents are good at — and where they struggle

Agents are strong at exactly the kind of work that wears people down: reading a lot of code quickly, following a repeatable process step by step, and working through a long list of small fixes without losing patience. A fiftieth file gets the same care as the first.

They're weaker where judgment and outside context matter. An agent can't read your mind, so a vague goal like "make this better" gives it too little to work with. It also can't know whether a decision is right for your business or your users — only whether the code runs. And it doesn't automatically carry today's conversation into tomorrow's. Something it needs to remember has to be written down, so a later Session can read it back.

Why you stay in charge

Handing real actions to a program on your computer only works if you trust what it's allowed to do. Claude Code builds that trust in with permissions.

Reading files to understand your project never needs your approval. But before Claude Code runs a command or changes a file, it stops and asks you first. You can approve it once, or tell it to always allow that exact action from then on, so it stops asking for the same thing again.

> **Try this:**
>
> read what an agent is about to do before you approve it. That single pause is where you're actually in charge.

Approving an action in the moment isn't the whole job. Read the changes an agent makes, run its tests, and check that the result actually meets the goal you gave it. Trust, but verify covers this in full.

What it costs: tokens and usage limits

None of this runs for free. Every time an agent reads a file, thinks through a step, or writes a reply, it's processing tokens — small pieces of text, each a few letters long. Tokens are the unit Claude Code's usage is measured in.

Your plan includes a usage allowance that renews on a regular schedule, and a meter that tracks how much of it you've used. A short, focused task uses a small slice of that allowance. A long conversation that drags on for hours, especially one juggling a lot of file content, uses far more — an agent re-reads its whole conversation every time it takes another step.

Keeping tasks focused, and starting a fresh Session when you move to unrelated work, is the easiest way to keep that meter moving slowly.

What you learned

- A chatbot only answers; an agent reads, writes, and runs commands to get a job done.
- An agent works in a loop — look, think, act, check, repeat — until the goal is met.
- Every agent has four parts: a model (brain), tools (hands), instructions (job description), and context and memory (notebook and desk).
- Agents are strong at repeatable, well-defined work, and weaker at judgment calls and vague goals.
- Permissions and reviewing the work keep you in charge, and tokens are the meter that tracks what it costs.

Next up: Claude Code in plain English

Claude Code in plain English

Claude Code is the command-line tool that does the real work: reading files, writing code, and running commands inside one project folder. This chapter walks through its core pieces — Sessions, house rules, skills, and more — in plain English, since every later chapter in this manual builds on these same words.

It lives in your project folder

Claude Code is a program you start from the terminal, and everything it does happens inside one project folder on your computer. It reads the files that are actually there, edits them, and runs commands, instead of guessing at a project it can't see.

Each time you start it, Claude Code begins a fresh conversation. That one back-and-forth, from a goal to a finished result, is a Session — this manual's word for one job you hand it. Session Manager keeps every Session on record, so you can reopen an old one and pick up where you left off instead of explaining the project again. That's called resuming, and it works because the full conversation history was saved the first time.

> **Try this:**
>
> resuming isn't just a Session Manager feature — Claude Code's own terminal supports it too. Run `claude --continue` to reopen your most recent conversation in a project folder, right where it left off.

House rules: instructions, settings, and permissions

A file named `CLAUDE.md` holds the standing instructions you'd otherwise repeat every conversation: your conventions, your build commands, what "done" looks like here. Claude Code reads it automatically at the start of every Session, so you only have to write a rule once.

> **Think of it like this:**
>
> CLAUDE.md is the employee handbook you hand a new hire on day one. It tells every Session your standing rules before it does anything else, so nobody has to ask twice.

Settings holds the rest of the machine-level choices: which commands run without asking, which outside tools are connected, and what model runs by default. It lives in a small set of files, each scoped to just you, to one project, or to your whole machine, so a rule can apply everywhere or only where you put it.

Permissions decide what Claude Code may do without stopping to ask you first. Reading a file to understand your project never needs approval. Before it runs a command or changes a file, it pauses and asks — and you can approve that one action, or tell it to always allow that exact thing from now on.

New tricks: skills and slash commands

A skill is a packaged set of instructions Claude Code can load for one kind of task: how your team reviews a pull request, how you deploy to staging, how a commit message should read. Write a clear description for it, and Claude Code loads the skill on its own whenever your request matches — no typing required.

A slash command works the other way: you type `/name` yourself, and only that loads it. Some slash commands ship with Claude Code, like `/model`; others are skills you or your team wrote, meant to be typed rather than left to fire on their own. Either kind lives as a file you can read, edit, and share.

Reaching outside: MCP Servers and Hooks

MCP Servers connect Claude Code to tools outside your project folder — a ticket tracker, a database, a design file — so it can read and act on real, current information instead of you pasting it in by hand.

Hooks are small scripts that run automatically at a specific moment: before a command runs, after a file changes, when a Session starts. Use one to enforce a rule Claude Code can't talk

itself out of, like blocking a risky command outright, or to react the instant something happens.

Building your own agent: subagents and choosing a model

A subagent is a specialized helper you define once — its own instructions, its own allowed tools, sometimes its own model — and call on for one particular kind of task. It works in its own space, so a long, noisy job, like reading through hundreds of test results, doesn't crowd out your main conversation.

Claude Code can run on more than one Claude model, and you choose which one a Session uses. A lighter, faster model suits a quick, well-defined task. A more capable model earns its keep on a hard problem that needs careful judgment. You can set a default and switch mid-conversation whenever the job changes.

Where to find each of these in Session Manager

Claude Code reads all of this straight from files on your computer — Session Manager doesn't invent a second copy of any of it. Each screen below is an editor over the real file, and each links to the chapter that covers it in depth.

Claude Code building block	Where it lives in Session Manager	Read more
Sessions & resuming	Sessions	Your first Session
CLAUDE.md house rules	System Prompt	House rules
Settings & permission rules	Settings, Permissions	House rules
Skills & slash commands	Skills	New abilities
MCP servers	MCP Servers	New abilities
Hooks	Hooks	New abilities
Custom agents (subagents)	Agent Library	Shaping your team
Choosing a model	Settings, Agent Library	Shaping your team

What you learned

- Claude Code runs from the terminal, working inside one project folder at a time.
- A Session is one conversation, and resuming reopens its full history instead of starting over.
- CLAUDE.md, Settings, and Permissions are Claude Code's house rules — what it always knows, and what it may do without asking.
- Skills load themselves automatically; slash commands only run when you type them.
- MCP Servers, Hooks, subagents, and your choice of model round out what Claude Code can reach, react to, and think with.

Next up: Meet your agency

Meet your agency — the team that works for you

Session Manager isn't one assistant. It's a small agency of AI agents, and you're the client who hires them. This chapter introduces the team, what each member actually does, and why splitting the work up beats handing it all to one assistant.

Your own small agency

Picture a small creative agency. A client brings an idea. A project lead turns that idea into a plan and hands out pieces of it. Builders each take one piece and finish it. Someone else checks the finished work before it ships.

Session Manager runs the same way, except every role on the team is an AI agent. You describe what you want. One agent plans it. Other agents build the pieces. One more checks the result. You approve the plan, and you get the final say on whether the work is actually done.

> **Think of it like this:**
>
> you're the client and the director of your own agency. You don't write the plan yourself, and you don't do the typing — you say what you want, and you sign off on what comes back.

Meet the team

Role	What they do	When you meet them
Architect	Plans the work with you and breaks it into small work orders. Never builds.	At the start of a Session, while you're still describing what you want.
Dev-lead	Builds one work order at a time, in the background.	While the Scheduler is running your work orders, usually without you watching.
Validator	Checks a finished plan once and gives a verdict on each work order.	After the last work order in a plan lands.
Project Home builder	Writes a one-page overview of a project.	Only when you ask for one.

The architect is the agent you actually talk to. It reads your project, asks a question when something's unclear, and turns a big idea into a series of small, well-defined work orders. It never writes the code itself — every work order it plans gets handed to a dev-lead instead. Because planning is worth taking slowly, the architect thinks with the most capable model available to it.

A dev-lead does exactly one work order, start to finish, with no view of the bigger plan. It reads the work order, makes the change, runs the checks it was given, and reports back. A dev-lead never decides what gets built or in what order — that call was already made by the architect before the work order reached it. Since a work order is already small and clearly scoped, a dev-lead runs on a faster, lighter model built for focused building rather than open planning.

The validator only appears once, at the end of a plan, after every one of its work orders has landed. It re-runs each one's own checks, compares the real result against what was promised, and gives a pass-or-fail verdict for each work order in the plan. It never edits your project and never adds new work orders of its own — it only reports what it found. Like a dev-lead, it runs on a fast model, because checking finished work is also a small, clear job.

A fourth agent, the Project Home builder, writes a single overview page for a project — what it is, how it's built, how to run it. It only runs when you ask for one; it never writes a page on its own.

How a job flows

Here's what happens between the moment you type a goal and the moment it's done:

You describe what you want, in your own words. → **Architect plans** the work and breaks it into small work orders. → **Work orders queued** — each one waits its turn, ready to run. → **Dev-leads build** them, one work order at a time, in the background. → **Validator checks** every finished work order against what was promised. → **You review** the result and decide what happens next.

You can keep working, or step away entirely, while the middle of that flow runs on its own. The work order queue and the Scheduler that runs it get their own chapter — this one is about who's doing the work, not how the calendar behind it works.

Why a team beats one assistant

A single assistant that plans, builds, and checks its own work has three problems a small team doesn't.

First, thinking is expensive, and typing shouldn't be. A careful plan is worth a slower, more capable model. Once that plan is broken into small, clear work orders, building each one doesn't need that same level of thought — a faster, lighter model finishes it just as well, for less. One assistant using its most capable setting for every step pays the planning price for typing work too.

Second, one assistant grading its own homework misses its own mistakes. The validator wasn't in the room while the work got built, so it checks with fresh eyes — the same reason a second person proofreads your writing better than you can.

Third, small tasks succeed more often than big ones. A work order with one clear goal is easy to check and easy to get right. A single sprawling task, done in one long pass, has more places to quietly go wrong — and no natural point to stop and check the result before moving on.

Your part in it

None of this runs without you. You still describe the goal, approve the plan before any work order runs, and read what comes back. The agency does the typing; you stay the director.

The four roles above are the ones Session Manager ships with, but they aren't the only agents you can have. You can write your own, with their own job description and their own rules, and add them to your team. Shaping your team covers how.

> **Try this:**
>
> next time the architect hands you a plan, read the list of work orders before you approve it. That's the moment you're actually directing.

What you learned

- Session Manager works like a small agency: you're the client and director, not the typist.
- The architect plans and breaks work into small work orders — it never builds.
- A dev-lead builds exactly one work order at a time, in the background, with no view of the bigger plan.
- The validator checks a finished plan once, gives a pass-or-fail verdict per work order, and never edits your project.
- A team beats one assistant: it saves thinking for planning, checks work with fresh eyes, and succeeds more often because each task stays small.

Next up: Your first Session — from idea to working agent

Your first Session — from idea to working agent

This chapter walks you from a blank screen to your agency's first finished job. You'll install the free app, open a project, and hand your team a goal in plain English. Ten minutes from now, you'll have watched an agent actually work.

What you need first

You need one thing before you start: Claude Code itself, installed and signed in. Session Manager doesn't replace Claude Code — it wraps it in a cockpit, so the CLI still needs to be authenticated with your account before anything can run. If you're not sure, open a terminal and run `claude login`.

Everything else is free. Session Manager itself never asks you to pay, sign up, or unlock a tier — you only need Claude Code ready to go. You don't need to know any command-line tricks, and you don't need to have written code before. The cockpit is built for a curious first-timer just as much as a veteran developer.

Install and take the tour

1. Go to `https://bilko.run/products/session-manager` and download the installer for your computer. On a Mac, pick Apple Silicon or Intel. On Windows, pick the .exe.
2. Open the downloaded file. On a Mac, drag the app into Applications. On Windows, run the .exe.
3. The first time you open it, your computer may warn you, because this preview build is not signed yet. On a Mac, right-click the app and choose Open. On Windows, click More info, then Run anyway.
4. On first launch, the Setup checklist offers one-click installs for anything missing, such as git and Claude Code. It also asks you to sign in to Claude.

5. The app opens on the Dashboard — one page listing every project and session on your machine.

On Linux, grab the AppImage from `https://github.com/StanislavBG/claude-code-session-manager/releases/latest`.

On that very first launch, a short guided tour opens on its own and points out the pieces of the cockpit you'll use most: where projects live, where sessions run, and where to change settings. Skim it or skip it — nothing about it is required. You can bring it back whenever you like: press `Cmd-K` (or `Ctrl-K` on Linux) to open the command palette, then choose "Restart guided tour."

Open a project

Session Manager works on one project folder at a time, and it keeps a separate tab for each one you open. Click "Open / Start Project" on the left, then pick the folder you want to work in. If the folder doesn't exist yet, the picker's New Folder button starts a brand new one.

> **Think of it like this:**
>
> a tab is a client file. Your agency can juggle several clients at once, but it never mixes up whose work belongs to whom — each project gets its own tab, its own history, its own sessions.

Opening a project you've already worked in just switches back to its tab — you never end up with two tabs for the same folder.

Give your agency its first job

Inside a project, open its Sessions view and click "+ New Session." A form opens asking for an Agent, a Mission, and a goal.

1. Pick an Agent — the "who." This is which member of your agency runs the session. The architect, the project lead you met last chapter, is a solid first pick for almost anything. Choose None to talk to plain Claude with no agent persona.

2. Pick a Mission — the "what kind of work." Feature is for building something new, Bug is for fixing something broken, and Discussion is for thinking something through before any code changes. Choose None and your goal is sent as written, with no task framing.
3. Describe the outcome you want, in your own words, in the goal box.
4. Click "Start Session."

The form also has a "Develop loop" checkbox. Leave it on for the normal planning workflow. Turn it off for plain Claude: no /develop skill and no queued plans, so that session cannot hand work to the scheduler.

Clicking "Start Session" sends your goal as the agent's very first instruction, and the session starts working right away.

> **Try this:**
>
> describe the result you want, not the steps to get there. "Add a dark mode toggle to Settings" gives the architect room to plan; "open this file and change line 40" doesn't leave it any planning to do. Naming a real constraint — a deadline, a file that must not change — helps more than naming a technique.

The life of a Session

Every session moves through the same three stages, in the same order, and it's always in exactly one of them.

Stage	What it means
Proposed	Set up but not started. Nothing has run, and nothing has been spent.
Active	Running. It's doing the work and reporting back.
Completed	Finished, with its full history kept for later.

Clicking "Start Session" moves a new session straight from Proposed to Active in one step. But sometimes a session sits as Proposed for a while first — if you close the form before

starting it, or if one of your agents files a suggestion of its own for you to decide on. A Proposed session shows two buttons: "Approve & start," which sends it exactly the way "Start Session" would, and "Discard," which cancels it but keeps the record.

Watch it work

Once a session is active, you can watch it two ways, and they're both views of the exact same conversation — nothing is duplicated or out of sync between them, and switching views never starts a second session by accident.

Chat reads like a conversation: your goal, the agent's replies, and its questions, one message at a time. It's the friendliest way to follow along, and it's where most first sessions are spent. Terminal shows the raw command line underneath, everything the agent is actually typing and running, for when you want to see every detail instead of a summary of it. Switch between them with the Chat and Terminal buttons at the top of the session. Most of the time, Chat is all you need — reach for Terminal when you want to see exactly what's happening underneath.

What you learned

- You need Claude Code installed and signed in before Session Manager can run anything.
- Install with one command, then replay the guided tour anytime from the command palette.
- "Open / Start Project" gives each project its own tab.
- "+ New Session" asks for an Agent (who), a Mission (what kind of work), and your goal, then "Start Session" sends it.
- Every session is Proposed, then Active, then Completed — and Chat and Terminal are two views of that one session, never two different ones.

Next up: Work that runs while you're away — plans and the Scheduler

Work that runs while you're away — plans and the Scheduler

A big idea doesn't run as one giant task. The architect turns it into small, written work orders, and the Scheduler runs them in the background, in order, while you do something else. This chapter shows you how that work order is written, and how the Scheduler decides what to run and when.

A work order is a written contract

When you ask the architect to build something, it doesn't just start typing. It writes a work order first — a short, plain document with four parts, so a dev-lead can pick it up later and know exactly what to do.

Part	What it holds
Goal	One sentence: what this piece of work is for.
Checklist	How to know the work is actually done — checks a dev-lead can run without you in the room.
Notes	Where to look, what already exists, anything the dev-lead shouldn't have to guess.
What not to do	The edges of the job — what's deliberately left for a later work order, or left alone entirely.

That checklist has to be something a dev-lead can check on its own, with no screen to look at and no mouse to click. "Open the app and try it" isn't a checklist item — a dev-lead can't do that. "Run this test and it passes" is.

> **Think of it like this:**
>
> a work order is a work ticket you'd hand a contractor who won't be in the room while they build — everything they need is on the ticket, because you won't be there to answer questions.

From your idea to a queue of work orders

You bring the architect an idea, in your own words — a bug to fix, a feature to add. The architect asks about anything unclear, then splits the idea into several small work orders, usually so each one is small enough to finish on its own. You'll typically hear this called `/develop` when you type it directly, but saying what you want works just as well — the architect is always the one doing the planning.

Each work order then joins the Scheduler's queue for your project, ready to run. Some work orders need another one to finish first — a work order can say it depends on an earlier one, and the Scheduler always waits for that dependency before starting it. Nothing else decides the order; two work orders with no dependency between them can run in any order, including at the same time.

Its own copy of your project

Each work order that runs gets its own copy of your project to work in, so two work orders running at the same time never step on each other's changes. When a work order finishes cleanly, its result is folded back into your real project.

> **Good to know:**
>
> this is why two work orders can safely run at once even when they touch the same project — each one is working in its own copy until it's done.

How many run at once

Every Claude Code job on your machine — Scheduler work and Chat conversations alike — draws from one shared, machine-wide limit, so your laptop is never asked to run more agents than it can actually handle. You can raise or lower that limit from the Home tab. Set it to zero and nothing new starts, without touching anything already running.

Claude Code itself has a usage limit that resets every few hours. When you're close to that limit, the Scheduler pauses on its own rather than firing work orders that would fail — and picks back up by itself once your usage window resets. You can also pause it yourself at any time.

A job's life

Here's what one work order goes through, from the moment it's written to the moment it's done:

The architect writes the work order and adds it to the queue. → **The Scheduler waits** for a free slot and for any dependency to finish first. → **A dev-lead runs it** in its own copy of the project, in the background. → **It reports back** — done, stuck, or with a question for you.

→ **Its result** is folded back into your real project.

Reading a work order's status

Every work order in the queue carries one status, in plain words:

Status	What it means
Pending	Waiting its turn — for a free slot, or for a dependency to finish first.
Running	A dev-lead is working on it right now, in its own copy of your project.
Completed	Done, and folded back into your project.
Failed	It ran and didn't finish successfully. The Scheduler will usually give it another try on its own.
Needs review	Stopped with a question only you can answer — not a failure.

Needs review is worth pausing on, because it's easy to mistake for a failure. It isn't one. A dev-lead reached a fork it genuinely can't choose for you — maybe the work order was unclear, or it found something that changes the plan — and it stopped rather than guess. That question comes back to you in the Session that wrote the work order, so you can answer it and let the work continue.

> **Watch out:**
> a work order stuck in Needs review isn't going anywhere until you answer it. If it's blocking others that depend on it, answering it promptly keeps the rest of the queue moving.

Taking the wheel: Pause, Resume, and Run now

Three controls in the Scheduler let you step in whenever you want:

1. **Pause** stops new work orders from starting. Anything already running keeps running — Pause never kills work mid-flight.
2. **Resume** clears a pause, whether you set it or the Scheduler paused itself on a usage limit, and lets the queue start firing again right away.

3. **Run now** fires the next ready work order immediately, instead of waiting for the Scheduler's own timing.

> **Try this:**
>
> open the Scheduler tab and look at a single work order's status. You'll see it move from Pending to Running to Completed — usually without you touching anything at all.

What you learned

- A work order is a written contract: a goal, a checklist, notes, and what not to do.
- The architect turns your idea into small work orders and queues them for you.
- The Scheduler runs them one after another, waiting only when a work order depends on an earlier one, each in its own copy of your project.
- A machine-wide limit caps how many run at once, and the Scheduler pauses and resumes on its own around your usage limits.
- Pending, Running, Completed, Failed, and Needs review — and Needs review is a question, not a failure.

Next up: Trust, but verify — how the work gets checked

Trust, but verify — how the work gets checked

An agent can sound completely sure of itself and still be wrong. This chapter shows you the checks your agency runs before it calls anything done — and the one check that's still yours to make.

Why an agent can be confidently wrong

An agent writes its answer one word at a time, each word chosen because it fits well after the last one. Nothing in that process checks the answer against reality as it goes. A wrong answer can come out reading just as smooth and certain as a right one.

This isn't a flaw you can nag away. It's how the technology works, for every agent, on every job. So your agency doesn't lean on any one agent trusting itself — it builds the checking in as a separate step, done by someone other than whoever wrote the code.

> **Think of it like this:**
>
> a contractor who's sure the wiring is right still gets it inspected. Confidence was never the test. A second, independent look is.

The checklist, written before the work starts

Every work order carries a checklist, written by the architect before a dev-lead touches any code. That order matters. Writing the checklist first means "done" was defined by the goal, not guessed backward from whatever got built.

A good checklist item is something a dev-lead can check on its own, with no screen to look at and no one in the room. "The button feels right" isn't checkable that way. "Running this command prints this result" is. That's why work orders lean on commands and tests, not impressions.

Tests: checks that run themselves

A test is a small program that checks another program. It runs a piece of code, compares what came out to what should have come out, and reports pass or fail — automatically, without a person watching. A dev-lead runs the tests its checklist names before calling a work order finished.

Tests catch a real class of mistake: code that looks right but breaks a case nobody thought to try by eye. They can't catch everything, though. A test only checks what it was written to check — which is one reason your agency doesn't stop at "the tests passed."

The validator's second look

Once every work order in a plan has landed, the validator — your agency's quality control — reviews the whole plan, one time. It doesn't take any dev-lead's word for it. It re-runs each work order's own checklist commands itself, then reads the actual result against what the work order promised.

The validator hands back one verdict per work order: verified, meaning the evidence backs up what was promised, or refuted, meaning it doesn't — with a plain reason why. Those verdicts go to the architect, the one who keeps track of the whole plan and decides what happens next.

When a work order stops to ask you something

Sometimes a dev-lead hits a fork it genuinely can't choose for you — the work order was unclear, or it found something that changes the plan. Rather than guess, it stops and marks the work order Needs review. That's a question, not a failure, and it waits for you in the Scheduler.

1. Open the work order's card and read what it's asking. The Review button takes you straight to the Session that wrote it, where the question is waiting.
2. Answer it there — clear up what was unclear, or point out what changed — then press Retry to send the work order back into the queue with your answer in hand.

3. Or set it aside. If you leave it, your agency won't wait on you forever: it will eventually skip that one work order on its own, so anything else in the queue that isn't waiting on it can keep moving.

> **Watch out:**
>
> "the tests pass" is not the same as "done." Tests only check what someone thought to write a test for. A work order can pass every test it names and still miss the point of what you actually asked for — that's exactly the gap the validator, and you, are there to catch.

You are the final judge

Checklists, tests, and the validator all exist to make your judgment easier, not to replace it. They filter out the obvious misses so what reaches you is worth your time. But whether the finished work actually solves your problem is still your call, not your agency's.

Look at what changed before you treat something as truly finished, especially for anything you care about getting right. A verified verdict is strong evidence. It isn't a substitute for you deciding the job is actually done.

What you learned

- An agent can sound certain and still be wrong — checking is a separate step, done by someone else.
- A work order's checklist is written before the work starts, so "done" means the goal, not whatever got built.
- Tests are automatic checks a dev-lead runs itself, but they only catch what they were written to catch.
- The validator reviews a whole plan once it's finished, re-running each checklist itself and handing back a verified or refuted verdict per work order.
- Needs review is a question, not a failure — read it, fix and retry, or set it aside and let your agency move on. Either way, you're the one who decides a job is truly done.

Next up: Shaping your team — Agents, Missions and one-click Actions

Shaping your team — Agents, Missions and one-click Actions

Every agent in your agency is just a plain text file — no coding needed to write one. This chapter shows you how to write your own, pick a Mission for it, and turn your favorite setup into a single button.

Write an agent in plain English

Open the Agent Library and you'll see a list of agents already on your machine — the architect, the dev-leads, and any others you've added. Each one is a small text file with a few labeled parts, called fields. You fill them in, and the agent is ready to work.

- **name** — the file's short label, all lowercase with hyphens, like `release-notes-writer`.
- **description** — one sentence saying what this agent does. It shows up in every list where you pick an agent, and it's read aloud to the agent itself at the start of a Session, so write it like an instruction, not a title.
- **instructions** — the longer text below the fields. This is the agent's own rulebook: what it should always do, what it should never do, and how it should think about the work.
- **model** — optional. Leave it blank and the agent uses whatever the Session would use anyway. Set it and every Session that hires this agent uses that model instead.

Here's a short example — an agent whose only job is turning a list of changes into a friendly note for users:

```
---
name: release-notes-writer
description: Turns a list of finished changes into a short, friendly note for use
model: sonnet
---

You write release notes for our users. Keep each note to two or three
```

```
sentences. Say what changed and why it helps them, and skip any detail
they'll never see.
```

A good agent has a description that reads like a real instruction, not a job title, and instructions that are specific enough to remove guesswork. "Write good code" tells an agent nothing it doesn't already assume. "Never touch a test file unless the test itself is wrong" tells it something it can actually follow.

> **Think of it like this:**
>
> the Agent Library is your agency's staffing directory. Every entry is a job description you write once. Any Session can hire that person for the work, and the description you wrote is the first thing they read before starting.

Picking a model — and what it costs

Session Manager offers a small set of models: Haiku, Sonnet, Opus, and Fable, plus the "match my Session" option that leaves the choice to whatever's already running. They aren't interchangeable on price. Haiku is the cheapest and fastest. Opus is the most capable and the most expensive per exchange. Sonnet sits in the middle and is the sensible everyday pick.

The rule of thumb built into the Agent Library itself is blunt: give a cheap model to an agent that only reads files and reports back, and save the expensive model for an agent that has to make real judgment calls — deciding what's broken, or what approach is safest.

> **Try this:**
>
> open an agent you already have, and check its model field. If it's blank, that agent is quietly inheriting whatever model its Session happens to use — which may be pricier than the work needs.

Your copy, or the project's copy

An agent you write lives on your machine and works in every project you open — it's your personal copy. But one project can keep its own small override of that same agent, changing only its model, without touching the version everyone else sees. That's useful when one project's work is genuinely heavier or lighter than usual, without renaming the agent or making a second one.

Missions — what kind of work a Session is

Every Session carries exactly one Mission, alongside its Agent. Where the Agent answers "who's doing this work," the Mission answers "what kind of work is it." You manage the list of Missions in the Tag Library.

Three Missions show up as buttons when you start a Session by hand:

Mission	What it sets up
Feature	Plans new work, then hands the actual building to the Scheduler as work orders.
Bug	Reproduces the problem first, then queues a fix — never guesses at the cause from a description alone.
Discussion	Keeps the Session talk-only. No files change until you decide something is worth building.

A handful of other Missions exist behind specific buttons elsewhere in the app — publishing a release, or writing a project's overview page — and you'll never need to pick those by hand.

Each Mission carries its own short opening instruction, which the agent reads before your own goal. Discussion's, for example, tells the agent plainly that the point is a decision, not a code change. That's why swapping Missions changes how a Session behaves, even with the same goal typed in and the same agent doing the work.

What you can edit from the Tag Library is which agents are meant for which Mission — tick a box next to an agent to connect it to a Mission, or remove it. What you can't do is invent a brand-new Mission from the app itself; the built-in list is fixed on purpose, so it stays small and every Session means the same thing by "Feature" or "Bug."

Actions — a favorite agent, one click away

Once an agent is written the way you like it, you can turn it into a button. Give the agent a project — one specific project, or every project — and a short opening instruction, and it appears as its own button in that project's Sessions toolbar, right next to "+ New Session."

Pressing that button isn't a shortcut around Sessions — it starts a brand-new one, the same way "+ New Session" does. It just skips the typing: the agent you scoped becomes the Session's Agent, its connected Mission becomes the Session's Mission, and the instruction you wrote becomes the opening goal. The button's label is the agent's name unless you give it a shorter one.

What you learned

- An agent is a plain text file with a name, a description, instructions, and an optional model — no code required.
- Models differ in price and power: cheap models for simple, read-only work; Opus for real judgment calls.
- An agent lives on your machine by default; one project can override just its model without touching the shared copy.
- A Mission sets what kind of work a Session is and what it reads before your goal; the built-in list is fixed, but which agents connect to which Mission is yours to edit.
- Scoping an agent to a project turns it into a one-click Action button — a shortcut to a normal new Session, never a second way to create one.

Next up: House rules — instructions, settings and permissions

House rules — instructions, settings and permissions

Tell every agent how you like things done, and decide what they may do without asking — without switching off the safety rails.

Claude Code reads a small set of files before it does anything else. Session Manager gives you editors for those files, so you never have to hand-edit them yourself. This chapter covers three of them: your machine-wide System Prompt, a project's own house rules, and the Settings and Permissions that shape how every Session behaves.

The System Prompt: your machine-wide house rules

The System Prompt tab lives on the Home face, under Configure. It edits one real file: `~/.claude/CLAUDE.md`. Every Session that Claude Code starts, in every project on this machine, reads this file first. Saving here changes what every agent on your machine already knows before it reads your actual request.

This tab has no scope switcher, because the file it edits has no scope to switch. If your house rules file uses `@path/to/file.md`-style imports to pull in other files, the tab lists each one in a left-hand rail so you can open and edit it too — they count as part of the same instructions the CLI loads.

> **Think of it like this:**
> the System Prompt is the handbook every new hire at your agency reads on day one, no matter which project they're assigned to. Change the handbook, and every future hire reads the new version.

A project's own house rules: CLAUDE.md

A single project can add its own rules on top of the machine-wide ones. Claude Code looks for a file named `CLAUDE.md` in the project's own folder, and a second file named

`CLAUDE.local.md` for rules that apply only on this machine. Both are read fresh whenever a Session starts in that project.

Session Manager doesn't give these two files their own Configure tab. Open them through File Explorer instead, the same way you'd open any other file in the project. `CLAUDE.md` is meant to be checked into the project's own version control and shared with anyone who works on it; `CLAUDE.local.md` stays on your machine only.

Settings: three scopes, one winner

The Settings tab, also under Configure, edits the app-wide preferences Claude Code keeps in a file called `settings.json` — theme, environment variables, and other machine-level choices. There isn't just one copy of this file. There are three, one per scope, and Settings lets you switch between them with a scope selector at the top of the tab.

Scope	File	Applies to
User	`~/.claude/settings.json`	Every project on this machine.
Project	`<project>/.claude/settings.json`	This project only. Meant to be shared with your team.
Local	`<project>/.claude/settings.local.json`	This project, this machine only. Never shared.

When the same setting is written in more than one file, Local wins over Project, and Project wins over User. The Settings tab has a Guided view that shows you the merged result across all three scopes at once, so you can see which value will actually take effect before you change anything.

> **Good to know:**
>
> Permissions, which we cover next, isn't a separate file at all. It's a section inside these same three `settings.json` files, just with its own tab and its own editor for that one section.

Permissions: fewer interruptions, same safety rails

By default, Claude Code stops and asks before doing anything risky — deleting a file, running an unfamiliar command, reaching outside your project folder. That's the safety rail, and the Permissions tab was never meant to remove it. What it does is let you tell Claude Code, once, which specific things are safe to do without asking again.

Permissions has three rule lists and one overall mode:

Rule list	What it does
Allow	Skips the prompt for anything matching the rule.
Deny	Blocks anything matching the rule, no prompt, no override.
Ask	Forces a prompt for anything matching the rule, even if it would otherwise be allowed.

Above the three lists sits one bigger switch: the default mode. It's the agent's overall posture before any single rule is even checked — options include the normal ask-when-unsure mode, a mode that accepts routine edits automatically, a read-only planning mode, and a mode that skips permission checks entirely. Most people never touch it and leave it on the normal, safest setting.

The tab opens on an Effective view, showing the merged rules from all three scopes together, before you ever see a raw list. That order is deliberate: check what's already true before you add a new rule that might duplicate — or contradict — one that's already there.

> **Watch out:**
>
> an Allow rule written at Project scope ships to everyone who works on that project, on their machine, the moment they pull your changes. A command you personally trust is not the same as a command your whole team should skip asking about. Keep personal convenience rules at User or Local scope, and save Project scope for rules the whole team has actually agreed on.

The rule of thumb: Settings is for everyone, Agents and Missions are for one job

Every file this chapter covers is read fresh by Claude Code before each Session starts, and none of them know which Session is about to run. A change in Settings or Permissions, at Project or User scope, changes behavior for every current and future Session in that scope — not just the one you're thinking about right now.

So if you only want one Session to behave differently, Settings is the wrong lever. Pick a different Agent for that Session, or give it a Mission that already carries the instructions you want. Save Settings and Permissions for what should genuinely be true no matter which Session runs, or no matter what kind of work it's doing.

What you learned

- The System Prompt tab edits `~/.claude/CLAUDE.md`, read by every Session on this machine.
- A project's own `CLAUDE.md` and `CLAUDE.local.md` add rules for that project, opened through File Explorer.
- Settings has three scopes — User, Project, Local — and Local beats Project beats User.
- Permissions holds Allow, Deny, and Ask rules that stop needless prompts without turning off the safety rail.
- A Settings or Permissions change reaches every Session in its scope; per-Session behavior belongs on that Session's Agent or Mission instead.

Next up: Giving your agents new abilities

New abilities — skills, plugins, tools and hooks

Teach your agents new tricks: packaged skills, plugins, connected tools and automatic hooks — and how to add them safely.

Your agency doesn't stop at whatever Claude Code knows out of the box. You can teach it new know-how, connect it to outside tools, and set up actions that fire on their own. Four Configure tabs cover this: Skills, Plugins, MCP Servers, and Hooks. Each edits a real file Claude Code already reads — nothing here is a parallel system layered on top.

Skills: packaged know-how

A skill is a set of written instructions for one kind of task, saved as a file. Some skills start on their own — Claude notices your request matches what a skill is for, and loads it without you asking. Others only run when you type a slash command, like `/develop`.

The Skills tab lists what's already on your machine, split into two groups: Skills and Slash Commands. Pick a skill and you can read it, and turn it off with a **Toggle** without deleting it — handy if a skill keeps firing when you don't want it to. Removing a skill moves its folder to the trash, so a change of mind later is still recoverable.

Switch the tab to **Library** and you get a searchable catalog of skills and plugins you don't have yet, instead of the ones you've already installed.

> **Think of it like this:**
> a skill is a training manual one of your agents keeps on a shelf. Some manuals they reach for automatically, the moment the job looks familiar. Others they only open when you hand them the exact page number.

Plugins: bundles from the Library

A plugin is a bundle — several skills, agents, or connected tools, packaged together and installed in one step. Open the Plugins tab and you land on **Installed**, a read-only view of every plugin already on your machine: its name, where it came from, and a drill-in showing exactly what it added. You can look, but you can't edit a plugin's contents from here — that keeps one clear place to edit a skill, instead of two competing editors for the same file.

To add a new one, switch to **Library**. Each row shows the plugin's name, a description, and an **Install** button that runs a real install for you, in the background, with progress shown right in the row.

Ability	What it is	When to use it
Skills	Packaged know-how for one kind of task, some auto-loaded, some typed as a slash command.	You want an agent to know how to do something specific, every time it applies.
Plugins	A bundle of skills, agents, or tools you install together from the Library.	You want a whole toolkit at once, not one skill at a time.
MCP Servers	Connections that let an agent reach outside tools and services.	The work needs data or an action outside your project folder.
Hooks	Small commands that run automatically when something happens in a Session.	You want the same action to happen every time, without asking each time.

MCP Servers: connecting outside tools

MCP stands for a protocol Claude Code uses to reach tools that live outside your project — a database, a ticket tracker, another app entirely. The MCP Servers tab lists every server you've registered, at either User scope (every project on this machine) or Project scope (this project only).

Adding a server to the list only proves the entry is well-formed. It says nothing about whether Claude Code can actually reach it. A misspelled command, a service that needs you to sign in

first, or a server still waiting on your approval all look identical on the page: present, and untried.

That's why the tab gives you a live check. Press **test connections** and it asks the real CLI which servers actually answered, then shows a status next to each one: Ready to use, Failed to connect, Needs you to sign in, or Not checked yet. Read that status before you start a Session that assumes a server's tools are there — a Session that starts before a server answers won't pick up its tools partway through.

> **Try this:**
>
> after adding or editing any MCP server, press
> **test connections**
>
> right away. A saved entry and a working connection are two different things, and only the check tells you which one you have.

Hooks: automatic actions on events

A hook is a small command that Claude Code runs by itself, triggered by something happening in a Session — before it uses a tool, after it stops, when a Session begins. The Hooks tab edits the exact setting Claude Code already reads, at whichever scope you choose.

Pick an event on the left, then write the command that should run and which situations should trigger it. Saving proves the command is well-formed. It proves nothing about whether the command actually works, whether it matches what you meant it to, or whether it exits cleanly — a typo in a hook can sit there silently doing nothing until the one day you needed it.

That's what the **test fire** button is for. It sends your command a sample event and shows you the real result — exit code, and everything it printed — before Claude Code ever runs it for real.

> **Watch out:**
>
> a hook is a command your machine actually runs, with real permission to touch files or the network. Session Manager always asks you to confirm before test-firing one, naming the exact command first. Only install skills, plugins, MCP servers, and hooks from sources you trust — a hook you didn't write and don't understand is not one to click through blindly, even in a test.

One rule that covers all four

Skills, Plugins, MCP Servers, and Hooks all do the same kind of thing: they give your agents more power than they started with. That power is worth having, but it's also worth checking. Read what a skill actually says before you trust it. Look at what a plugin adds before you install it. Test an MCP server's connection before you build work around it. Test-fire a hook before you let it run unattended.

None of these four tabs invent a system of their own — every one of them edits a real file Claude Code already reads on its next launch. Session Manager gives you a clear way to see what's there, add to it safely, and prove it works before you rely on it.

What you learned

- Skills are packaged know-how — some load themselves, some you type as a slash command.
- Plugins bundle several abilities together; Installed is read-only, Library is where you install from.
- MCP Servers connect outside tools; always test connections before trusting one is live.
- Hooks run commands automatically on Session events; test-fire one before you trust it in production.
- Only install skills, plugins, MCP servers, and hooks from sources you trust — they all run real commands with real permissions.

Next up: A tour of the whole cockpit

Around the cockpit — Home, memory, usage, history and more

Session Manager has more than one screen, and each one earns its place. This chapter is a short walk through the rest of the cockpit — where to check on everything at once, what your agents remember, how to watch your usage and spending, and how to publish a page for the world to see.

Home and Project Home

Home

Home is the Dashboard — a one-page overview of every project and every Session on your machine. Open it and you'll see how many projects you have, which Sessions need your attention, which ones are actively running, and a short list of the ones you touched most recently.

Think of Home as your front desk. You don't do your work there — you just check it on your way in, so nothing important is waiting for you without your knowledge.

Project Home

Switch into a single project's tab, and its Home becomes Project Home instead: an auto-written overview of just that project, kept as two libraries. **Knowledge Pages** are written overviews; **Video Pages** are short demo videos. Each press of a button makes one new entry, and you can rename or delete any entry from its list. Press again later, and a fresh entry catches up with whatever changed.

File Explorer

File Explorer is the built-in file browser and editor. Open it from Home, and it starts at your own computer's home folder. Open it from inside a project tab instead, and it starts right

there. Browse the folder tree on one side, and open any file to read or edit it on the other — no separate app required.

Memory and the Clusters report

Memory

Memory holds the notes an agent keeps and can read back in a later Session — things it learned about your project, your preferences, or work still in progress. Open Memory for a project, and you'll see every note saved so far, split by whether it belongs to the whole project or to one particular agent.

The Clusters report

A pile of separate notes is hard to skim. The Clusters tab groups a project's memory into a handful of named, related bundles, so you can see the shape of what your agents know at a glance instead of reading every note one by one. It's built from notes already saved on your machine, so opening it costs nothing — press Refresh only when you want it to look for new patterns.

Usage meters and History

Usage meters

Claude Code's usage limits reset on a schedule, and the usage meters show you exactly where you stand against them: one meter for your rolling five-hour session window, and separate meters for your weekly limits — across all models, and broken out for Opus, Sonnet, and any connected app. Each meter shows a percentage used and a countdown to its next reset.

> **Think of it like this:**
> a usage meter is a fuel gauge, not a speedometer. It won't slow you down — it just tells you how much room is left before a window resets, so a long work session never catches you by surprise.

History

History is a record of past Sessions, with cost and usage over time, gathered across every project on your machine. Pick a time range and a measure — spend, tokens, active time, and more — and History draws the trend, ranks your projects against each other, and lets you drill into any one of them for the detail behind the total.

Voice input and the recording indicator

Voice input

Voice lets you talk to your agents with your microphone instead of typing. Your speech is transcribed on your own machine and turned into text, the same as if you'd typed it yourself.

The speech model only loads when you first press the microphone, so that first press takes a few seconds while the voice panel says *Loading speech model*. After half an hour without using voice, Session Manager unloads it to free memory, and the next press loads it again.

The recording indicator

Whenever your microphone is capturing audio, a red bar appears at the very top of the window and says so in plain words. It's deliberately impossible to hide or cover up — a promise that you'll always know when Session Manager is listening, with nothing happening quietly in the background.

Simple mode and telemetry

Simple mode

Sometimes you don't want a cockpit at all — you just want one terminal. Launch with the `--simple` flag, and Session Manager boots straight into a single terminal in whatever folder you launched it from, with none of the navigation, tabs, or configuration screens around it. Just you and Claude Code.

Anonymous telemetry

Session Manager reports anonymous usage data back to bilko.run by default — never your prompts, your files, or anything that could identify you, only things like which app version you're running and roughly how the app gets used. It's on so we can see whether the app is working well for people. It is never required for the app to work for you.

> **Good to know:**
>
> nothing sent home carries your name, your email, your file paths, or anything from inside a Session — the whole point of this data is that it can't be traced back to you.

Turning it off takes one step. In Settings, find the telemetry toggle and switch it off, or dismiss the first-run notice with its own "Turn off" button. Running Session Manager from a script or a scheduled job instead? Set the environment variable `SM_TELEMETRY=0`, and it overrides everything else, every time.

The web remote: drive all your computers

You can work with Session Manager from a phone or any browser, and you can pair several computers to it, such as your laptop and a Windows PC at home. Each computer is paired once. After that, you pick which one to drive from the header of the web app. Everything travels through the bilko.run relay, scrambled so that only your browser and your computer can read it.

1. On the computer, install and open Session Manager. Go to Settings, then Phone remote, and turn on **Allow remote control from the web**.

2. To let the browser type commands, such as running `hostname` in a terminal, also turn on **Allow command writes**. Leave it off if you only want to watch.
3. On any phone or browser, open `bilko.run/projects/session-manager` and sign in. Tap **Generate pairing code**. Back on the computer, choose **Pair Device...** and enter the 8-character code.
4. The first time a browser connects to a computer, the web app waits for you to confirm a 6-digit code on that computer (Settings, then Phone remote). Confirm it once per browser, per computer.
5. To add another computer, repeat steps 1 to 4 on it. To switch, use the computer switcher in the web app header and pick an online computer.

> **Good to know:**
>
> the 6-digit code check is what stops anyone in the middle from posing as your computer. A paired computer can be forgotten from the web app at any time, and its command-write switch can be turned off whenever you like.

What you learned

- Home is the Dashboard for your whole machine; Project Home is the same idea for one project.
- File Explorer browses and edits any file without leaving the cockpit.
- Memory keeps what your agents have learned, and the Clusters report groups it into readable bundles.
- Usage meters track your five-hour and weekly limits; History tracks cost and usage over time.
- Voice lets you talk to your agents, the recording indicator always tells you when it's listening, and telemetry is anonymous, on by default, and one setting — or `SM_TELEMETRY=0` — away from off.
- The web remote lets a phone or browser drive every computer you pair; pair each once, confirm the 6-digit code, then switch from the header.

Next up: Good habits — great results, safely

Good habits — great results, safely

Ten habits that make your agents more helpful, cheaper and safer — and the common mistakes to avoid.

None of these habits need a new button or a new setting. They're the tools already covered in this manual, used in the order that gets you the best result with the least risk. Read them once and they start to feel automatic.

Habits that keep your agency effective

1. Give every job its own Session

A Session is one job, from a goal to a finished result — not a running conversation you keep repurposing for whatever comes up next. A fresh Session per job keeps its history honest: anyone reading it later, including you, sees exactly what it was asked to do, and nothing else mixed in.

2. Describe the outcome, not the steps

Tell the architect what you want to be true once the work is done, not which file to open or which line to change. "Add a dark mode toggle to Settings" gives it room to plan; a list of clicks doesn't. A real constraint — a deadline, a file that must not change — helps more than naming a technique.

3. Let the architect plan before anyone builds

The architect turns a big idea into small, well-defined work orders before any code gets written. Skipping that step to save time usually costs more of it later, once a dev-lead hits a decision only the plan should have made.

4. Keep work orders small

A work order with one clear goal is easy to check and easy to get right. A sprawling one has more places to quietly go wrong, with no natural point to stop and check it along the way. If a work order feels big, it's usually two work orders wearing one name.

Habits that keep you in control

5. Answer Needs review questions promptly

Needs review is a question, not a failure — but it's also a stop sign. Any work order waiting on it can't move until you answer. Leaving it for days doesn't only delay one job; it can quietly stall everything queued behind it too.

6. Check the result yourself

Checklists, tests, and the validator filter out the obvious misses, but whether the finished work solves your actual problem is still your call. A verified verdict is strong evidence. It isn't a substitute for looking at what changed.

7. Watch usage and cost

Every Session and every work order spends real usage against your Claude Code plan. The usage meters and cost history around the cockpit show you where that's going, so a plan running hotter than expected gets noticed in an afternoon, not a month.

Habits that keep you safe

8. Keep passwords and keys out of prompts and files

Never type a real password, API key, or token into a goal or a CLAUDE.md file or a Project-scope setting. CLAUDE.md is meant to be checked into version control and shared with your whole team, and a Project-scope setting ships to everyone the moment they pull your changes. A secret belongs in a credential store built for that job, never in plain text an agent reads or writes.

9. Keep your project under version control

Every work order the Scheduler runs works in its own copy of your project, and making that safe copy depends on your project already being tracked in git — the version-control system most software projects use. Without git, a work order runs directly in your project instead of a protected copy. Git also gives you a plain undo button: it can show you exactly what changed and step you back to before it happened.

When an agent goes in circles

Sometimes a Session keeps retrying the same fix, or drifts further from what you meant with every reply. That's your cue to stop, not to keep clarifying in place.

10. Stop and restart with a clearer goal

Rewriting the goal mid-conversation rarely helps — the agent is already anchored to everything said before it. It's usually faster to end that Session and start a new one with a tighter goal, written the way you would if this were the first time you'd ever asked for it.

Common mistakes to avoid

Most trouble traces back to one of these habits running in reverse:

- Treating "the tests passed" as the same as "done," instead of checking the result yourself.
- Letting a Needs review question sit for days while everything behind it waits too.
- Pasting a real secret into a goal or a shared file to get something working quickly.
- Writing one giant work order because splitting it felt like more effort, then untangling what went wrong somewhere inside it.
- Working outside version control, so a mistaken change has nothing to undo back to.

Watch out:

a password or key typed into a prompt, a CLAUDE.md file, or a Project-scope setting doesn't stay private. Any agent in that Session can read it back — and because CLAUDE.md and Project settings are meant to be shared, so can anyone else who works on the project. Keep secrets in a proper credential store, never in text an agent reads or writes.

Think of it like this:

these habits are the same ones that make any contractor relationship go well — a clear brief, one job at a time, checking the invoice, and keeping your own paperwork in order. Working with agents doesn't change any of that. It only means the habit matters faster, because the work happens faster too.

What you learned

- Give each job its own Session, and describe the outcome you want, not the steps to get there.
- Let the architect plan, and keep work orders small enough to check on their own.
- Answer Needs review promptly, check the finished result yourself, and watch usage and cost as you go.
- Keep passwords and keys out of prompts and files, and keep your project under version control so any change can be undone.
- If an agent goes in circles, stop and start a new Session with a clearer goal instead of patching the old one.

Next up: Glossary — every term in plain English

Glossary — every term in plain English

Every word this manual uses, explained in a sentence or two, with a pointer to where it is covered.

Active

A Session that is running: it is doing the work and reporting back to you. See: Your first Session — from idea to working agent

Agent

The member of your agency who works a Session — the "who" behind the job. See: Shaping your team — Agents, Missions and one-click Actions

Agent Library

The place where you browse, edit, and create the agent personas on your machine. See: Shaping your team — Agents, Missions and one-click Actions

AI agent

Not a chatbot that just answers. It is a program that plans, uses tools, and does multi-step work toward a goal you give it. See: What is an AI agent?

Approve & start

The button that turns a Proposed Session into an Active one and sends its first instruction. See: Your first Session — from idea to working agent

Architect

Your agency's project lead. It plans the work, breaks it into pieces, and tracks it to done. See: Meet your agency — the team that works for you

Chat

The conversation view of a Session, where you type a message and read the reply. See: Your first Session — from idea to working agent

Claude Code

The command-line tool from Anthropic that Session Manager wraps in a cockpit. See: Claude Code in plain English

Completed

A finished Session, archived with its full history. See: Your first Session — from idea to working agent

Dashboard

The one-page overview of every project and every Session on your machine. See: Around the cockpit — Home, memory, usage, history and more

Dev-lead

One of your agency's builders. It takes a single, already-scoped piece of work and builds exactly that. See: Meet your agency — the team that works for you

File Explorer

The built-in file browser and editor. See: Around the cockpit — Home, memory, usage, history and more

History

A record of past Sessions, with cost and usage over time. See: Around the cockpit — Home, memory, usage, history and more

Hooks

Small scripts that run automatically when something happens in a Session. See: New abilities — skills, plugins, tools and hooks

House rules

This manual's word for the standing instructions, settings, and permissions you give every Session. See: House rules — instructions, settings and permissions

MCP Servers

Connections that let an agent reach outside tools and services. See: New abilities — skills, plugins, tools and hooks

Memory

Notes an agent keeps and can read back in a later Session. See: Around the cockpit — Home, memory, usage, history and more

Mission

What kind of work a Session is — the "how." It shapes how eagerly your agency queues follow-up work. See: Shaping your team — Agents, Missions and one-click Actions

Needs review

A work order that stopped because it has a question only you can answer. See: Trust, but verify — how the work gets checked

New Session

The button that starts a fresh Session. See: Your first Session — from idea to working agent

Permissions

The rules that decide what an agent may do without stopping to ask you first. See: House rules — instructions, settings and permissions

Plugins

Installable extensions that add new skills, agents, or tools to Claude Code. See: New abilities — skills, plugins, tools and hooks

Project Home

A one-page, auto-written overview of a single project. See: Around the cockpit — Home, memory, usage, history and more

Proposed

A Session that has been set up but has not started yet. Nothing has run, and nothing has been spent. See: Your first Session — from idea to working agent

Queue

The list of work orders that are waiting, running, or finished. See: Work that runs while you're away — plans and the Scheduler

Scheduler

Your agency's work calendar. It lines up work orders and runs them, even while you're away. See: Work that runs while you're away — plans and the Scheduler

Session

One job you hand to your agency, from a goal to a finished result. Chat and Terminal are two ways of looking at the same Session. In the code, a Session is called an Epic — you will not see that word in the app itself. See: Your first Session — from idea to working agent

Session Manager

This app: a desktop cockpit for running several Claude Code agents at once. See: Welcome — your friendly map of what's ahead

Settings

App-wide preferences, such as theme, voice, and other machine-level choices. See: House rules — instructions, settings and permissions

Skills

Packaged instructions an agent can load for a specific kind of task. See: New abilities — skills, plugins, tools and hooks

System Prompt

The house rules every Session reads before anything else — your standing instructions file. See: House rules — instructions, settings and permissions

Tag Library

The place where you manage the list of Missions a Session can carry. See: Shaping your team — Agents, Missions and one-click Actions

Terminal

The raw command-line view of a Session, for when you want to see everything it is doing. See: Your first Session — from idea to working agent

Validator

Your agency's quality control. It checks finished work against what was promised. See: Meet your agency — the team that works for you

Voice

Talk to your agents with your microphone instead of typing. See: Around the cockpit — Home, memory, usage, history and more

Work order (PRD)

One scheduled piece of work the Scheduler runs on its own, with everything it needs to finish written down. See: Work that runs while you're away — plans and the Scheduler

---

© Bilko.run · The Session Manager Field Manual is free, updates included — re-download the latest at bilko.run/manual.